BRAC University — Department of Computer Science & Engineering

CSE 470: Software Engineering Project Solution

Assignment Topic: System UML Class Diagram & Object-Oriented Architecture Specification
Project Title: TrafficEase BD — Smart Urban Traffic Management & Commuter Bypass Navigation Platform

Assignment Question:
Design a Class Diagram for your CSE 470 Project system.
Instructions:
• Identify key classes
• Define relevant attributes
• Define operations/methods
• Clearly represent relationships: Associations, Inheritance, Aggregation or Composition where applicable.

1. System Architecture & Class Overview

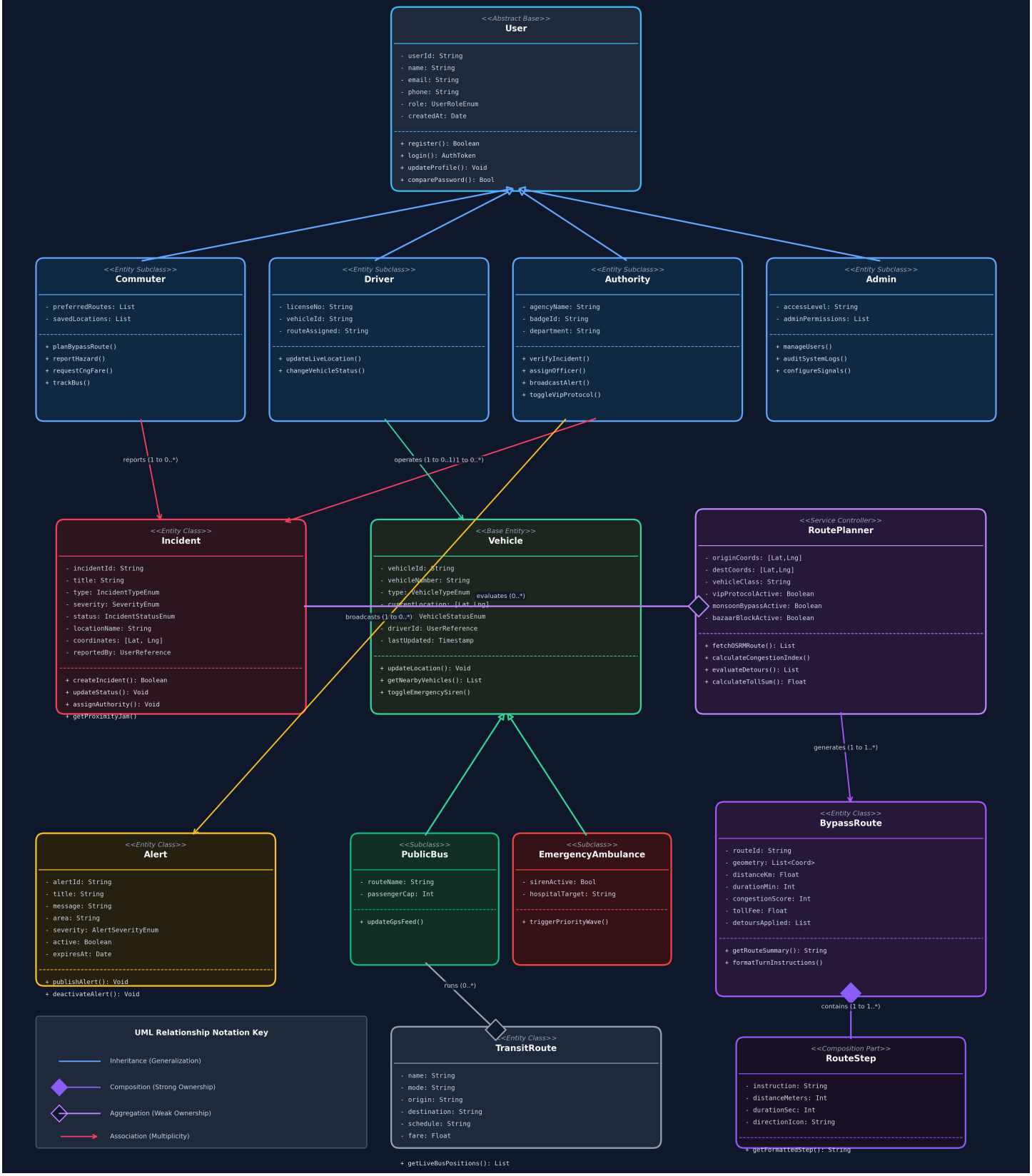
The **TrafficEase BD** platform provides real-time traffic monitoring, incident verification, and intelligent bypass route planning for Dhaka commuters. The object-oriented architecture is divided into three functional packages: **User & Identity Management**, **Incident & Asset Management**, and the **Smart Bypass Navigation Engine**.

2. Visual UML Class Diagram

The diagram below illustrates all primary entities, methods, visibility scope (+ public, - private), and explicit UML relationships.

TrafficEase BD — System UML Class Diagram

BRAC University | CSE 470: Software Engineering Project Architecture



3. Detailed Key Class Specifications

Class Name & Stereotype	Attributes (Name: Type)	Operations / Methods	Description / Role
User «Abstract Base»	- userId: String - name: String - email: String - phone: String - role: UserRoleEnum - createdAt: Date	+ register(): Boolean + login(): AuthToken + updateProfile(): Void + comparePassword(): Bool	Abstract superclass encapsulating core user identity, credentials, and authentication logic.
Commuter «User Subclass»	- preferredRoutes: List - savedLocations: List	+ planBypassRoute() + reportHazard() + requestCngFare() + trackBus()	General public commuter using route navigation, fare estimation, and crowdsourced reporting.
Authority «User Subclass»	- agencyName: String - badgeId: String - department: String	+ verifyIncident() + assignOfficer() + broadcastAlert() + toggleVipProtocol()	Traffic police or municipal authority responsible for verifying incidents and issuing alerts.
Driver «User Subclass»	- licenseNo: String - vehicleId: String - routeAssigned: String	+ updateLiveLocation() + changeVehicleStatus()	Transit driver transmitting real-time GPS telemetry feeds.
Incident «Domain Entity»	- incidentId: String - title: String - type: IncidentTypeEnum - severity: SeverityEnum - status: IncidentStatusEnum - coordinates: [Lat,Lng]	+ createIncident() + updateStatus() + assignAuthority() + getProximityJam()	Core entity representing accidents, roadworks, waterlogging, or congestion points.
Vehicle «Base Entity»	- vehicleId: String - vehicleNumber: String - type: VehicleTypeEnum - currentLocation: [Lat,Lng] - status: StatusEnum	+ updateLocation() + getNearbyVehicles() + toggleEmergencySiren()	Represents fleet units active on the road (buses, CNGs, emergency vehicles).
RoutePlanner «Service Controller»	- originCoords: [Lat,Lng] - destCoords: [Lat,Lng] - vehicleClass: String - vipProtocolActive: Bool - monsoonBypassActive: Bool	+ fetchOSRMRoute() + calculateCongestionIndex() + evaluateDetours() + calculateTollSum()	Control class coordinating route calculations, OSRM queries, and dynamic detour insertions.
BypassRoute «Domain Entity»	- routeId: String - geometry: List - distanceKm: Float - durationMin: Int - congestionScore: Int - tollFee: Float	+ getRouteSummary() + formatTurnInstructions() + calculateTollSum()	Represents calculated path output containing geometry, jam load, and step instructions.
RouteStep «Part Class»	- instruction: String - distanceMeters: Int - durationSec: Int - directionIcon: String	+ getFormattedStep(): String	Granular turn-by-turn driving maneuver step belonging directly to a BypassRoute.
Alert «Domain Entity»	- alertId: String - title: String - message: String - area: String - severity: AlertSeverityEnum	+ publishAlert() + deactivateAlert()	Broadcast alert message distributed to commuters in targeted congested zones.
TransitRoute «Domain Entity»	- name: String - mode: String - origin: String - destination: String - schedule: String - fare: Float	+ getLiveBusPositions()	Defines public transit line corridors (e.g., Raida Bus, Metro Line 6).

4. Object-Oriented Relationships & Specifications

Relationship Type	Source Class	Target Class	Multiplicity & Explanation

Inheritance (Generalization)	Commuter, Driver, Authority, Admin	User	Subclasses inherit base user credentials and authentication methods, extending role-specific behaviors.
Inheritance (Generalization)	PublicBus, EmergencyAmbulance	Vehicle	Subclasses specialize base Vehicle attributes with transit route names or emergency siren triggers.
Composition (Strong Part-Whole)	BypassRoute	RouteStep	**1 to 1..***. RouteStep objects are created as integral components of a BypassRoute. If BypassRoute is destroyed, its steps cease to exist.
Composition (Strong Part-Whole)	TransitRoute	TransitStop	**1 to 1..***. Stops form the structural path sequence of a TransitRoute. Stop lifecycle is bound to the route container.
Aggregation (Weak Part-Whole)	RoutePlanner	Incident	**1 to 0..***. RoutePlanner references active Incidents to calculate proximity detours, but Incidents exist independently in the system database.
Aggregation (Weak Part-Whole)	TransitRoute	Vehicle	**1 to 0..***. PublicBus vehicles operate along a TransitRoute corridor, but vehicles can be reassigned without destroying the TransitRoute.
Association (Bi-directional / Direct)	Commuter	Incident	**1 to 0..***. Commuters create and submit incident reports (crowdsourced pothole/flooding pins).
Association (Direct)	Authority	Incident	**1 to 0..***. Traffic Authorities review, verify, assign officers, and change incident resolution status.
Association (Direct)	Authority	Alert	**1 to 0..***. Authority users broadcast emergency traffic warning alerts to active commuters.
Association (Direct)	RoutePlanner	BypassRoute	**1 to 1..***. RoutePlanner generates calculated BypassRoute instances for commuter selection.

5. Architectural Justifications & Mongoose ODM Mapping

• **Mongoose Schema Mapping:** In our Node.js/Express backend (backend/models/), persistent entities such as User, Incident, Vehicle, Alert, and TransitRoute translate directly to MongoDB collection schemas with 2DSphere geospatial indices.

• **Service & Controller Separation:** The RoutePlanner class functions as a pure controller/service, executing OSRM routing requests, evaluating spatial proximity using distance formulae, and dynamically injecting bypass waypoints for VIP protocol blocks or waterlogged streets.

• **Encapsulation & Security:** Passwords in User are hashed using bcryptjs via pre-save hooks, ensuring credential fields remain private and protected behind authorization middlewares.